

RhirePro - Completion Plan & Claude Prompt

Google for Startups Cloud (Start tier) and NVIDIA Inception readiness. Status is based on the code in the repository (main + branch claude/dreamy-clarke-511efn), checked on the build date. This replaces the earlier PDF, whose "what is built" section described an older Express/Mongo prototype.

How to use this document

1) Read the status tables (pages 1-2). 2) Run the two SQL files and set the environment variables (end of page 2). 3) Copy the prompt from page 3 onward (also saved as docs/CLAUDE_COMPLETION_PROMPT.md - copy from that file to avoid PDF line-break artefacts) into a fresh Claude Code session in this repository. The prompt makes Claude audit first, so it will not rebuild things that already exist.

Track A - Core product completeness

Feature	Status	Evidence / what remains
Job model + CRUD	VERIFY	Jobs table, status workflow, expiry and applicant-count triggers exist. Confirm dashboards read DB rows, not mock arrays.
Application model + APIs	VERIFY	Applications table, rejection stages, status workflow exist. Confirm apply / withdraw / status end to end.
Recruiter candidate search	VERIFY	Master search service, Boolean search, fuzzy match exist. Confirm Candidates tab is live data.
Notification backend	PARTIAL	Notifications tables, email functions exist. Add missing triggers; confirm in-app notifications.
Subscription / billing	VERIFY	Razorpay create-order + verify-payment, plans, usage limits exist. Add tests for untested paths.
Admin / moderation	PARTIAL	Super-admin area exists. Add job moderation queue (ties to fraud detection).
Rate limiting (OTP, reset, login, AI)	DONE	supabase/rate_limits.sql + netlify/shared/rateLimit.mjs. Run the SQL.
Security headers	DONE	nosniff, HSTS, frame, referrer in netlify.toml.
Content-Security-Policy	TODO	Add as Report-Only first (Razorpay / Supabase / Google sign-in).
Input validation / sanitisation audit	TODO	Audit every function and HTML-rendered user text.
Refresh-token rotation	VERIFY	Supabase Auth dashboard setting; verify and document.

Track B - AI/ML differentiators

Feature	Status	Evidence / what remains
LLM fit score + explanation	DONE	/api/match-score (OpenAI, cached in ai_match_scores).
Skills / certification insights	DONE	/api/ai-insights (OpenAI).
Embeddings backend (pgvector)	DONE	supabase/ai_embeddings.sql, /api/embed-sync. Run the SQL.
Semantic search + applicant ranking API	DONE	/api/semantic-search. UI not wired yet.
Resume parsing API (PDF)	DONE	/api/resume-parse. UI review screen not built yet.
Resume parsing UI (review + confirm)	TODO	Seeker profile screen.
DOCX resume support	TODO	Server-side docx-to-text, then model.
Wire embed-sync + backfill script	TODO	After profile save / job post; one-off backfill.
Smart search + recommendations + Best-match sort	TODO	Frontend wiring over the API.

Feature	Status	Evidence / what remains
AI job-description generator	TODO	New endpoint + recruiter UI.
AI career assistant (RAG)	TODO	New endpoint + chat panel.
Fraud / fake-job detection	TODO	Rules + embedding similarity + LLM; feeds moderation queue.
Video-interview analysis (NVIDIA Riva/NIM)	ROADMAP	Do not claim as built. Write the roadmap doc only.

Track C - Credibility

Item	Status	Owner / note
Terms, Privacy, Refund pages	PARTIAL	Exist. Add AI-processing disclosure (Claude prompt C1).
Honest AI copy on landing page	TODO	Claude prompt C2, after Track B.
Pitch deck + application answers	TODO	Claude drafts (C4); founders fill numbers.
Incorporation date, entity, reg. number	YOU	Decides eligibility for both programs.
Live site + email on your own domain	YOU	Google checks the match. Gmail will not work.
Google Cloud billing account ID (18 chars)	YOU	Needed before applying to Google.
Traction numbers, founder bios	YOU	Pilot users, waitlist, jobs posted.
Verify current program terms	YOU	NVIDIA and Google official pages; terms change.

Setup you must do first

Run in the Supabase SQL editor: supabase/rate_limits.sql and supabase/ai_embeddings.sql. Set in Netlify: OPENAI_API_KEY (optional OPENAI_MODEL, default gpt-4o-mini), SUPABASE_SERVICE_ROLE_KEY. GEMINI_API_KEY is no longer used. The seeker dashboard still names some functions "Gemini" internally (cosmetic only).

Suggested 5-day order

Mon: setup + Track A audit (A1-A7). Tue: B1, B3, B4, B5 (AI visible in the UI). Wed: B2, B6, C1, A7 CSP. Thu: B7, B8, C2, C4. Fri: C5 QA, then apply to Google (Start tier) first and NVIDIA Inception second. If time runs short, B7 and B8 are the first to cut; B1-B5 are the minimum for a truthful "AI-matched" claim.

The prompt - paste into Claude Code

Copy everything below the line. Source of truth: docs/CLAUDE_COMPLETION_PROMPT.md

You are working in the RhirePro repository (React 18 + Vite + TypeScript + Tailwind frontend; Supabase Postgres with RLS and SQL migrations in /supabase; serverless backend in /netlify/functions with helpers in /netlify/shared; Razorpay payments; Vitest tests in /src/__tests__). Read docs/GRANT_READINESS_PLAN.md first.

GOAL

Finish the product so it can truthfully be presented to Google for Startups (Cloud, Start tier) and NVIDIA Inception: close the gaps in Track A (core completeness), Track B (AI/ML differentiators) and the engineering parts of Track C (credibility). Work through the items below IN ORDER.

HARD RULES

1. AUDIT BEFORE BUILDING. Older notes claim the backend is Express/Mongo with no jobs, applications, billing or admin. That is WRONG. For every item, first read the code and decide: DONE / PARTIAL / MISSING. Report the verdict with file paths, and only build what is partial or missing. Never rebuild what exists.
2. AI provider is OpenAI. Use the shared helpers in netlify/shared/ai.mjs (embedText, generateJson, OPENAI_KEY). Key is OPENAI_API_KEY (server-side only, never VITE_-prefixed, never sent to the browser). Do not add Gemini calls.
3. Netlify synchronous functions die at 10 s. Keep every model call inside a budget and degrade gracefully (cached/stale result, or a clear JSON error).
4. Every AI or costly endpoint must: authenticate the caller (getCaller in netlify/shared/ai.mjs), authorise ownership (a user may only act on their own profile; a recruiter only on their own jobs/applicants), build prompts server-side from database rows (no caller-supplied free text sent to the model except a bounded search query), and call enforceRateLimit from netlify/shared/rateLimit.mjs.
5. Do not break payments (netlify/shared/payments.mjs arithmetic is pinned by tests), RLS policies, or existing routes. New SQL goes in /supabase as a new idempotent file (safe to re-run), with a header explaining WHY. Never edit an old migration.
6. Be honest in product copy and docs: only claim features that are live. Video-interview analysis (NVIDIA Riva/NIM) is ROADMAP, not shipped.
7. Never commit secrets. Never push to main; work on the current branch, commit in small logical commits, run `npx vitest run` and `npx vite build` before each commit, and fix failures instead of skipping tests.
8. Match surrounding code style and comment density. Add Vitest tests for new pure logic and for each endpoint's auth/validation/rate-limit paths (mock fetch).
9. After each item, update the status table in docs/GRANT_READINESS_PLAN.md.

ALREADY BUILT (verify, do not redo)

- Rate limiting: supabase/rate_limits.sql + netlify/shared/rateLimit.mjs, wired into send-otp, verify-otp, send-reset-otp, reset-password, super-admin-login, ai-insights, match-score, embed-sync, semantic-search, resume-parse.
- Baseline security headers in netlify.toml (no CSP yet).
- Embeddings backend: supabase/ai_embeddings.sql (pgvector, 768-dim, match_jobs / match_applicants RPCs), /api/embed-sync, /api/semantic-search.
- /api/resume-parse (PDF only; returns parsed JSON, saves nothing).
- /api/match-score and /api/ai-insights (now OpenAI).
- Terms, Privacy and Refund pages; Razorpay plans; super-admin area.

=== TRACK A: CORE COMPLETENESS ===

- A1. Jobs: confirm a recruiter can create, edit, pause, close and list a persisted job end to end from the UI (RecruiterDashboard, jobService). Fix anything that still renders mock arrays instead of database rows.
- A2. Applications: confirm a seeker can apply, withdraw, and see status; a recruiter can move status. Fix gaps and any mock data.
- A3. Recruiter candidate search/filter: confirm the "Candidates" tab uses real data (masterSearchService / boolean search). Replace any static sample data.
- A4. Notifications: confirm in-app notifications and transactional emails fire for: application received, status changed, job match, plan events. Build the missing triggers; make sure toasts are backed by real events.
- A5. Billing: confirm Razorpay order creation, signature verification, plan activation, subscription usage limits and cancellation work. Add tests for any untested path. Do not change pricing arithmetic.
- A6. Admin/moderation: confirm super-admin can review and remove jobs, suspend recruiters, and see reports. Add job moderation (flag, hide, reason, audit log entry) if missing.
- A7. Security hardening: (a) audit every netlify function for missing auth, missing input validation and over-broad service-role use; fix. (b) Add a Content-Security-Policy in netlify.toml as Content-Security-Policy-Report-Only first (allow Razorpay, Supabase, Google sign-in). (c) Verify Supabase auth refresh-token rotation/reuse detection is enabled and document the dashboard setting. (d) Sanitise all user-supplied text rendered as HTML. (e) Re-run supabase/verify_rls_security via scripts/verify_rls_security.mjs if possible.

=== TRACK B: AI/ML DIFFERENTIATORS ===

- B1. Resume parsing UI: in the seeker Profile area add "Parse my resume". It calls /api/resume-parse with the stored resume path, shows the extracted fields in a REVIEW screen (checkbox per section: skills, experience, education, certifications, headline, about), and only writes what the user confirms. Handle errors (unsupported type, too large, rate-limited) with clear messages.
- B2. DOCX support: extend resume-parse to accept .docx (convert to text server-side with a small dependency such as mammoth, then send text to the model). Enforce size limits.
- B3. Embedding sync: call /api/embed-sync after profile save and after job create/edit (fire-and-forget, never block the save). Write scripts/backfill-embeddings.mjs to embed all existing profiles and jobs in batches with a delay and resumability.
- B4. Semantic job search: replace the placeholder search bar behaviour with a "Smart search" mode calling /api/semantic-search {mode:"jobs", query}. Hydrate results through the existing RLS-protected job queries, show similarity as "Best match", fall back to keyword search on any failure.
- B5. Recommendations and applicant ranking: seeker "Recommended for you" uses semantic-search with no query; recruiter applicant list gets a "Best match" sort via {mode:"applicants", job_id}, with the existing /api/match-score explanation on demand.
- B6. AI job-description generator: recruiter enters title + 3-6 bullets; new endpoint /api/generate-job-description returns a structured draft (summary, responsibilities, requirements, skills) the recruiter edits before posting. Server-side prompt, rate-limited, never auto-publishes.
- B7. AI career assistant: new endpoint + chat panel for seekers using retrieval over the seeker's own profile and relevant active jobs (use match_jobs for retrieval). Capabilities: skill-gap analysis, interview prep, resume tips. Ground answers in retrieved rows, cap history length, rate-limit per user, refuse off-topic requests, never reveal other users' data.
- B8. Fraud / fake-job detection: new endpoint run on job create (and available to super-admin) that scores risk using rules (suspicious domains, requests for payment, off-platform contact, duplicate descriptions via embedding similarity) plus an LLM judgement. Store score and reasons in a new table; high-risk jobs go to the moderation queue instead of auto-publishing. Super-admin UI to review.
- B9. (Roadmap only) Do NOT implement video-interview analysis. Create docs/NVIDIA_ROADMAP.md describing the planned architecture using NVIDIA Riva (speech-to-text) and NIM microservices, GPU needs and cost estimate, clearly labelled "planned".

=== TRACK C: CREDIBILITY (ENGINEERING PARTS) ===

- C1. Update Privacy Policy and Terms to disclose that profiles and resumes are processed by third-party AI providers (OpenAI) for parsing, matching and search; describe retention, deletion and user rights; keep wording factual and flag it for legal review.
- C2. Landing page copy: describe AI features accurately (only what is live after Track B). Add a short "How our AI works" section.
- C3. Add a public /status or /about page and a visible contact email; make sure the site has no placeholder text and loads over HTTPS with the security headers.
- C4. Generate docs/PITCH_DECK.md (slide-by-slide text: problem, solution, product, AI architecture diagram in text, market, traction placeholders, team placeholders, the ask) and docs/APPLICATION_ANSWERS.md (Google for Startups Cloud and NVIDIA Inception form answers) using only verified facts from the repo. Mark every number you cannot verify as "[FOUNDER TO FILL]".
- C5. Final QA: write docs/QA_CHECKLIST.md and walk the demo path: sign up -> upload resume -> parse -> confirm -> embed -> smart search -> apply -> recruiter ranks applicants -> match explanation. Fix anything broken.

THINGS ONLY THE FOUNDERS CAN DO (list them, do not fake them)

Incorporation details, domain and domain email, Google Cloud billing account ID, real traction numbers, founder bios, verifying current NVIDIA and Google program terms on their official pages.

OUTPUT FORMAT

Start by printing the audit table (item, verdict, evidence). Then implement item by item. After each item print: what changed, files touched, tests added, how to verify manually. End with a final summary of what is done, what is deferred, and the founder checklist.